SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

BATTHINA JAYANTHI (VTU25786)

10211CS224
Full Stack Application Development
Slot : E3-B9

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”SMART CAMPUS EVENT MANAGEMENT SYSTEM” by BATTHINA JAYANTHI (VTU25786) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hafizur Rahman

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(BATTHINA JAYANTHI)

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management System is a web-based application designed to streamline the creation, management, and registration of campus events in universities and colleges. It provides a centralized platform where students can securely register, explore upcoming events, apply advanced search and filtering options, and manage their event participation. Administrators are provided with dedicated tools to create, update, and delete events, monitor registrations, and analyze participation through real-time statistics. The system incorporates modern technologies such as a React-based frontend and a Spring Boot backend with RESTful APIs, ensuring efficient communication and scalability. By replacing manual event management processes, the system minimizes errors, avoids overbooking through capacity constraints, saves time, and enhances overall user experience and operational efficiency.

Keywords: Campus Events, Event Management System, Spring Boot, REST API, MySQL, CRUD Operations, Web Application, Registration System, Automation

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
Auth	Authentication
CRUD	Create, Read, Update, Delete
DB	Database
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
MVC	Model View Controller
REST	Representational State Transfer
UI	User Interface

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

In the era of digital transformation, educational institutions are increasingly adopting smart solutions to streamline administrative and academic activities. One of the key areas that still relies on semi-manual processes is campus event management, which includes event creation, registration, and participation tracking. Traditional methods such as manual registrations, spreadsheets, and notice boards are time-consuming, error-prone, and inefficient when handling large numbers of students.

The Smart Campus Event Management System is developed as a modern web-based solution to overcome these challenges. It provides a centralized platform where students can explore, filter, and register for events, while administrators can efficiently manage event creation, monitor registrations, and analyze participation data.

The system leverages a full-stack architecture using modern technologies, ensuring scalability, responsiveness, and ease of use. It enhances transparency, reduces manual workload, prevents issues like overbooking through capacity constraints, and improves overall user experience for both students and administrators.

1.2 Aim of the Project

The primary aim of the Smart Campus Event Management System is to design and develop a user-friendly and efficient web-based platform for managing campus

events and student participation. The project focuses on achieving the following objectives:

- To automate the event management process within educational institutions
- To provide a centralized platform for students to explore and register for events
- To enable administrators to efficiently create, update, and manage events
- To reduce manual errors and prevent overbooking using capacity constraints
- To improve overall efficiency, accessibility, and user experience

1.3 Scope of the Project

The Smart Campus Event Management System focuses on developing a web-based platform that enables efficient management of campus events and student participation within educational institutions. The system allows students to explore available events, apply advanced search and filtering based on criteria such as event name, date, and department, and register for events online while ensuring capacity constraints are maintained to prevent overbooking. It also provides administrators with tools to create, update, and delete events, as well as monitor all student registrations and analyze participation through basic statistics. The platform is designed using a modern full-stack architecture with a React-based frontend, a Spring Boot backend, and an MYSQL database[6] for data storage. The system aims to reduce manual effort, improve accuracy, and streamline event coordination, while being scalable for future enhancements such as notifications, analytics, and integration with other institutional systems.

1.4 Framework

The system framework consists of multiple layers including the user interface, application logic, and database. The frontend layer is developed using web technologies such as HTML, CSS, and Bootstrap to provide a simple and interactive user experience. The backend layer is built using Spring Boot, which handles business logic and processes user requests. The database layer uses a MySQL database[6] to store event and registration data efficiently, ensuring reliable and persistent data management.

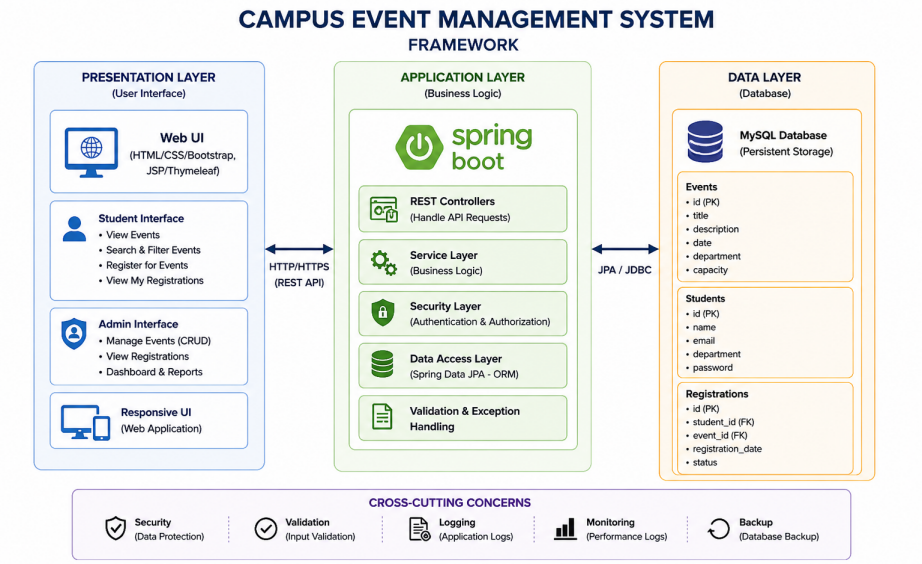

Figure 1.1: Framework of Event Management System

Figure 1.1 shows the framework of the Campus Event Management System, including the presentation, application, and data layers. It highlights user interaction through a web interface, backend processing using Spring Boot, and data storage using MySQL, along with features like validation and logging.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several event management applications are widely used in the market, such as Eventbrite, Meetup, and Townscript [7]. These platforms provide features like event creation, online registration, and user-friendly interfaces that enhance the overall experience for users and organizers.

In addition, modern web-based systems are developed using backend frameworks such as Spring Boot [4] and relational databases like MySQL [6] to efficiently manage event data and user information. Platforms like Whova and Bizzabo offer advanced functionalities such as attendee engagement, event analytics, networking tools, and automated communication, helping manage large-scale events efficiently.

Within universities, institutional portals are also used for managing internal events. These systems provide basic functionalities such as event listing and manual registration. However, they often lack scalability, modern user interfaces, and advanced filtering options.

Most existing applications are designed for large-scale public events such as conferences and exhibitions. They typically include features like ticket booking, payment integration, and analytics dashboards.

However, these platforms do not fully support campus-specific requirements such as department-based events, restricted access, and simplified workflows. This creates a need for a system specifically designed for academic environments.

Table 2.1: Comparative Analysis of Market Alternatives

Feature	**BookMyShow**	**Eventbrite**	**Townscript**	**Your Project**
Event Creation	Yes	Yes	Yes	Yes
User Registration/Login	Yes	Yes	Yes	Yes
Campus/Department Events	No	No	No	Yes
Event Listing	Yes	Yes	Yes	Yes
Registration Management	Yes	Yes	Yes	Yes
Payment Integration	Yes	Yes	Yes	No
Event Analytics	Yes	Yes	Yes	Basic
Admin Control	Limited	Limited	Limited	Yes
Direct Admin-User Communication	Limited	Limited	No	Yes

The Table 2.1 compares existing event management platforms with the proposed system. While all platforms support basic features like event creation and registration, they lack support for campus-specific requirements such as department-based events and direct admin-user interaction. The proposed system addresses these gaps effectively.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Campus Event Management System is developed using basic web technologies such as HTML, CSS, and Bootstrap. These technologies are used to design a responsive and user-friendly interface. The development is carried out using tools like Visual Studio Code, while web browsers are used for testing and debugging the application.

3.1.2 Structure of the Project

The frontend follows a structured approach where different HTML pages are used to represent various functionalities of the system. These pages include the home page for displaying events, login page for authentication, registration page for user input, and event form page for adding new events. Styling is handled using CSS to improve the visual appearance of the application. The pages are interconnected through links and form submissions, enabling seamless navigation and user interaction.

3.1.3 Execution Procedure

The frontend is executed as part of the Spring Boot application and is accessed through a web browser using a local server. Once the application starts, users can interact with the system by viewing events, registering for events, and performing

actions based on their role. The interface updates dynamically based on user input and system responses.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the system is developed using Spring Boot, which provides a robust framework for building web applications and handling business logic. Java Development Kit (JDK) is required for development, and Maven is used for managing dependencies and building the project. The application runs on an embedded server, which simplifies deployment and execution. Development is carried out using Eclipse IDE, and tools like Postman can be used for testing API functionality.

3.2.2 Structure of the Project

/src/main/java/com/campus/eventmanagement: Contains the main backend source code of the application.

/controller: Includes controller classes such as EventController and RegistrationController, which handle HTTP requests and manage user interactions.

/service: Contains business logic related to event management and registration processing.

/repository: Includes interfaces like EventRepository and RegistrationRepository that interact with the MySQL database using Spring Data JPA.

/model: Defines entity classes such as Event and Registration, representing database tables.

-

/src/main/resources: Contains configuration files and frontend resources.

/templates: Includes HTML pages such as index.html, login.html, register.html, form.html, and registrations.html used for user interface.

/static: Contains static resources used in the application.

/css: Includes styling files such as style.css for improving UI design.

/js: Contains JavaScript files for client-side functionality (if used).

/images: Stores images and assets used in the application.

/application.properties: Contains configuration settings such as MySQL database connection, server port, and JPA properties.

/src/test/java: Contains test classes for unit and integration testing.

pom.xml: Defines project dependencies such as Spring Boot Starter Web, Spring Data JPA, Thymeleaf, MySQL Connector, and build configurations.

EventmanagementApplication.java: Main class that serves as the entry point of the Spring Boot application.

3.2.3 Execution Procedure

The backend application is executed using Spring Boot, which starts an embedded server and listens for client requests. When a request is received, it is processed through the controller and service layers, and the necessary operations are performed. The backend communicates with the database to store or retrieve data and sends the response back to the frontend through HTTP requests.

3.3 Database Implementation

3.3.1 Database Configuration

The system uses a MySQL database for storing application data in a persistent manner. The database is configured in the Spring Boot application through the ap-

plication properties file, where details such as database URL, username, password, and driver class are specified. This configuration allows the application to establish a connection with the database during runtime.

3.3.2 Schema Structure

The database follows a relational structure in which data is organized into tables with defined relationships. Each table contains specific attributes, and relationships between tables are maintained using primary keys and foreign keys. This ensures data consistency and integrity across the system.

3.3.3 Tables created

The core functionalities are supported by the following tables:

- **Student Table:** Stores student details such as ID, name, email, department, and password for authentication.
- **Event Table:** Contains event information including title, date, department, type, description, and capacity.
- **Registration Table:** Acts as a link between students and events, storing details such as registration ID, student information, and event reference.

3.4 System Technology Stack Overview

Table 3.1: System Technology Stack

Component	**Technology**
Frontend	HTML, CSS, Bootstrap
Backend	Spring Boot, Spring Data JPA
Database	MySQL
API Communication	HTTP
Tools	Eclipse, VS Code, MySQL Workbench, Gite

The Table 3.1 provides a comprehensive summary of the frameworks and technologies utilized in the development of the Smart Campus Event Management System, categorized by their architectural role.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Campus Event Management System follows a layered architecture consisting of the presentation layer, application layer, and data layer. The presentation layer provides a web-based interface through which users interact with the system to view events, register, and perform administrative actions. The application layer is implemented using Spring and is responsible for handling business logic, processing user requests, and managing communication between the user interface and the database. The data layer uses a MySQL database to store event details and registration information in a structured manner.

The system ensures smooth data flow by processing user requests through controllers, services, and repositories before interacting with the database. This layered approach improves maintainability, scalability, and organization of the application. Basic validation and role-based access control are implemented to ensure that only authorized users can perform specific actions such as adding or deleting events. The overall architecture supports efficient operation, easy updates, and future enhancements of the system.

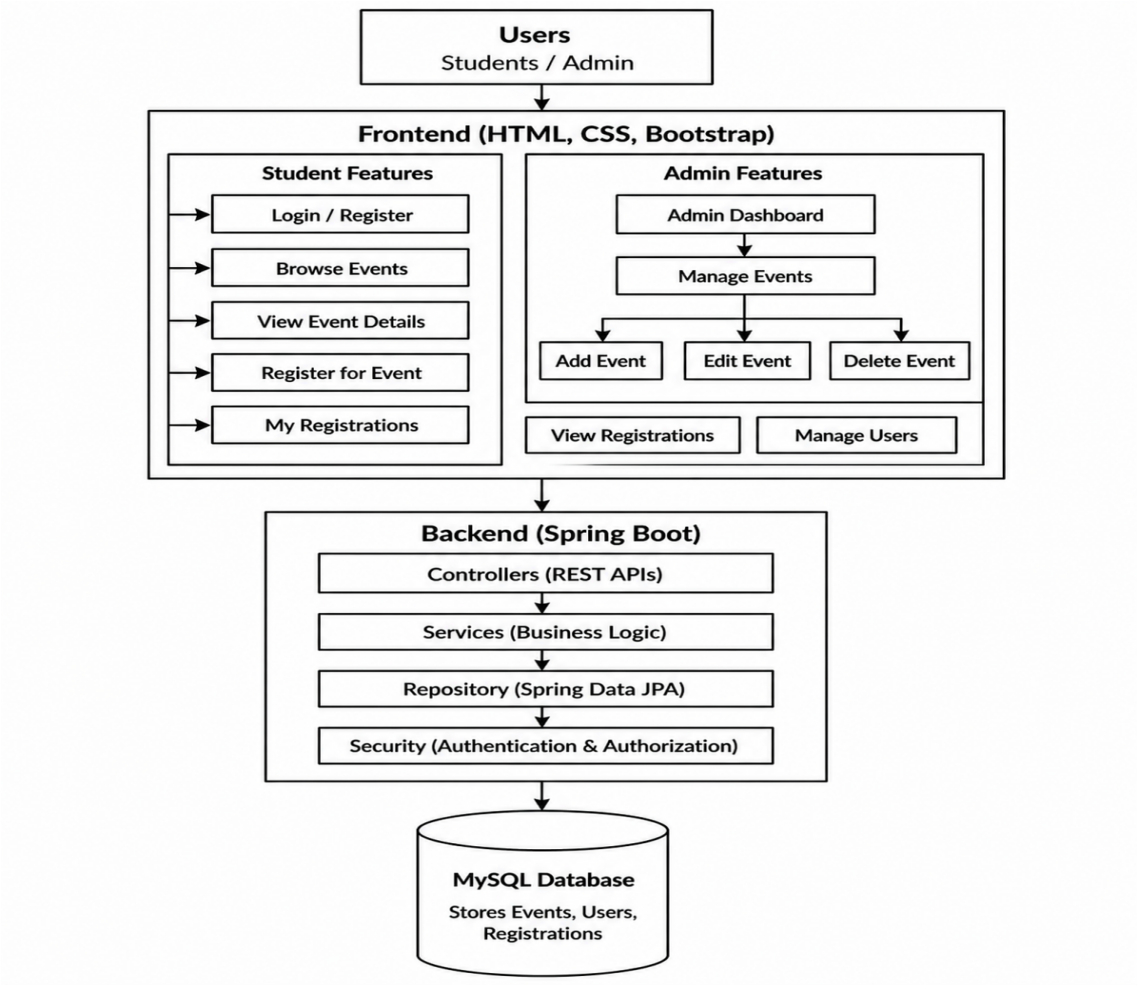

Figure 4.1: System Architecture of Event Management System

Figure 4.1 illustrates the layered architecture of the Campus Event Management System, which separates the presentation layer (HTML, CSS, Bootstrap-based UI), application layer (Spring Boot REST services), and data layer (MySQL database). It ensures efficient event management by enabling smooth communication between users and the system. The architecture supports secure data handling, scalable design, and seamless processing of user interactions, business logic, and database operations.

4.2 DataFlow Diagram

The Data Flow Diagram illustrates how data moves within the system. User inputs, such as login details or event registration requests, are sent from the frontend to the backend through API calls. The backend processes these requests, performs necessary validations such as checking event capacity, and interacts with the database to store or retrieve information. The processed data is then sent back to the frontend, which displays outputs such as event lists, registration confirmations, and user dashboards.

4.2.1 DFD Level 0: Context Diagram

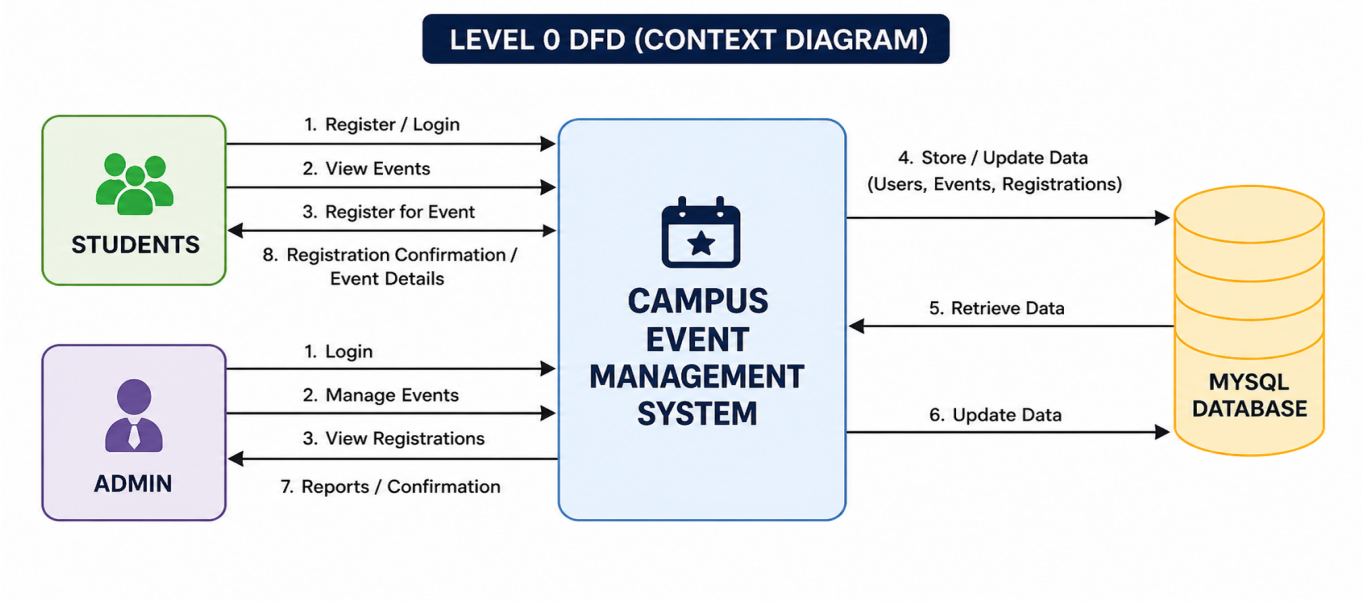

Figure 4.2: Data Flow Diagram Level 0: Context Diagram

Figure 4.2 says that the Level 0 Data Flow Diagram represents the overall system as a single process interacting with external entities such as students and administrators. It shows how user inputs like login, event requests, and registrations are processed by the system and how responses are returned through the database.

4.2.2 DFD Level 1: Functional Decomposition

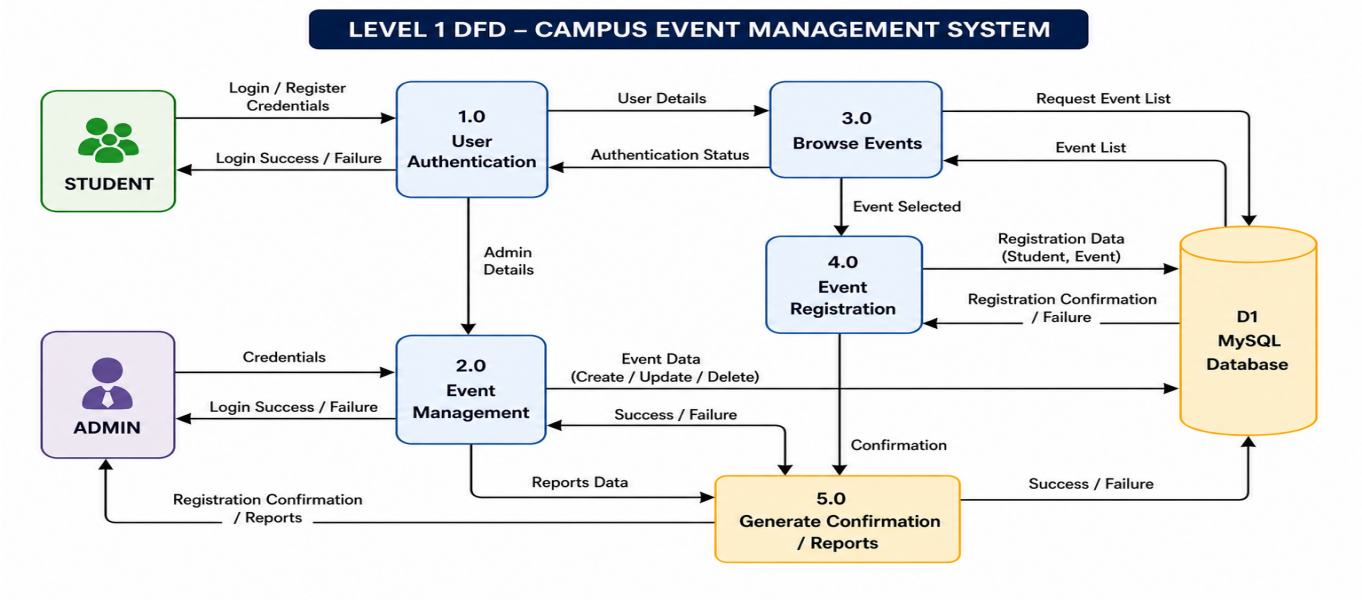

Figure 4.3: Data Flow Diagram Level 1: Functional Decomposition

Figure 4.3 says that the Level 1 Data Flow Diagram provides a detailed breakdown of the internal processes of the system, including user authentication, event browsing, event management, and event registration. It illustrates how data flows between these processes and the database, ensuring proper validation, storage, and retrieval of information for efficient system operation.

4.3 System Modules

The Campus Event Management System is divided into key modules such as User Management, Event Management, and Event Registration to ensure organized and efficient functioning of the application. Each module is responsible for handling specific operations including authentication, event handling, and student participation. This modular design improves system performance, enhances scalability, and makes the system easier to maintain and extend in the future.

4.3.1 Module 1: User Management

The User Management module is responsible for handling student and admin authentication processes including registration and login. It ensures that only authorized users can access the system and perform specific actions based on their roles. Student users can register and log in to browse and participate in events, while admin users can securely access management functionalities. The module stores user credentials in the database and validates login information during authentication. It also helps maintain session control and ensures data security within the system.

4.3.2 Module 2: Event Management

The Event Management module is designed specifically for administrators to manage campus events effectively. It allows admins to create new events, update event details, and delete events when required. Each event includes information such as event name, department, type, date, and description. This module ensures that all events are properly organized and updated in real-time, providing accurate information to users. It plays a crucial role in maintaining control over all event-related activities within the system.

4.3.3 Module 3: Event Registration

The Event Registration module enables students to view available events and register for them easily. It provides functionality to display event details and allows users to participate in selected events. The system validates each registration request to avoid duplicate entries and ensures proper linking between students and events. After successful registration, the system stores the details in the database and reflects them in both student and admin views. This module simplifies the participation process and improves user experience.

4.4 Advantages and Limitations of the Project

The Campus Event Management System offers multiple advantages by automating the event management process and reducing manual intervention. It improves efficiency by allowing administrators to manage events easily and enables students to access and register for events conveniently. The system ensures accurate data handling and provides a smooth interface for both users and administrators.

4.4.1 Advantages

- **Time Saving:** Automates event management and registration processes, reducing manual effort.
- **Error Reduction:** Minimizes human errors and prevents issues such as duplicate registrations and overbooking.
- **Easy Management:** Enables administrators to efficiently manage events and track student participation.

4.4.2 Limitations

The system relies on internet connectivity, and its performance may be affected during network issues. Initial development and setup require technical knowledge of technologies such as Spring Boot and database management. Additionally, regular maintenance is necessary to ensure smooth functioning and to handle updates or system improvements over time.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Implementation Overview

The Smart Campus Event Management System is designed as a comprehensive web-based platform integrating multiple modules for both students and administrators to efficiently manage campus events and participation.

- Landing Page Interface
- Authentication Module
- Student Dashboard
- Admin Dashboard
- Event Management Module
- Event Registration Module
- Advanced Search & Filtering
- My Registrations (Student Panel)
- Registration Management
- MySQL Database Implementation

5.2 System Interface Visualizations

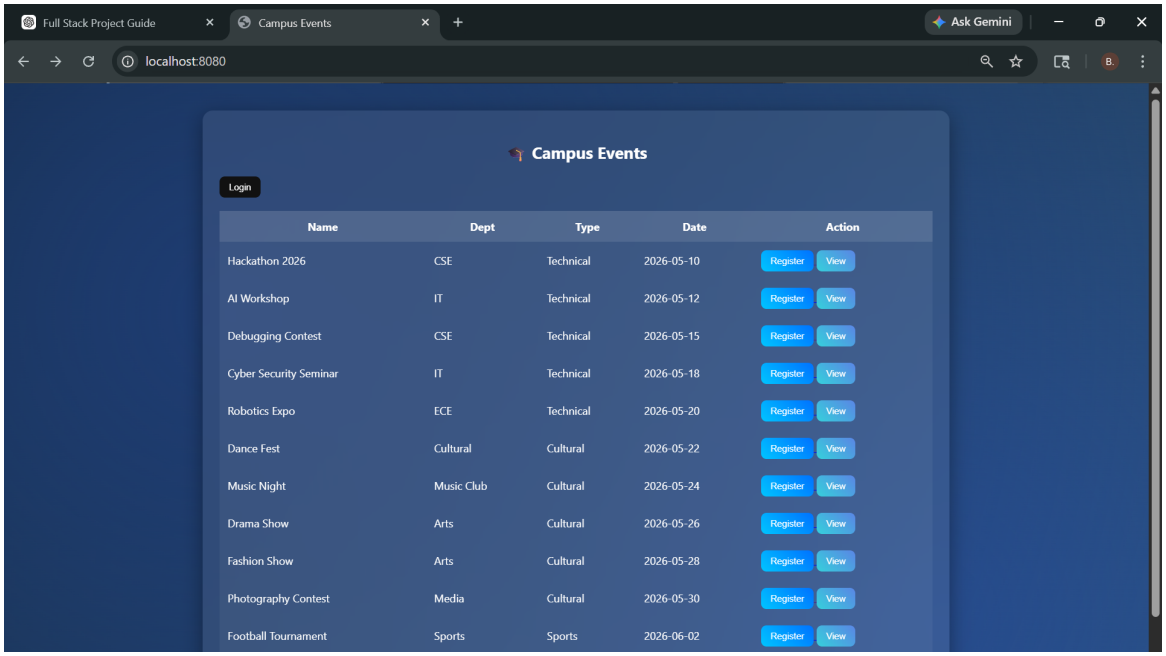

Figure 5.1: Landing Page Interface

Figure 5.1 says that to Discover and register for campus events like workshops, seminars, and networking sessions with ease.

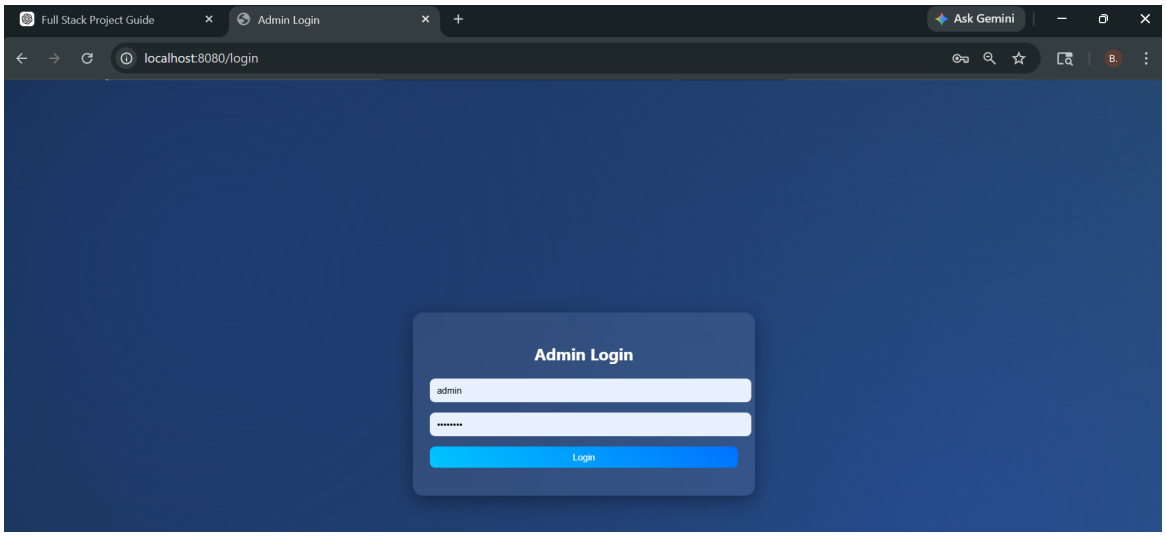

Figure 5.2: Admin Login Interface

Figure 5.1 allows administrators to securely log in using their credentials to access system functionalities such as event management and monitoring registrations.

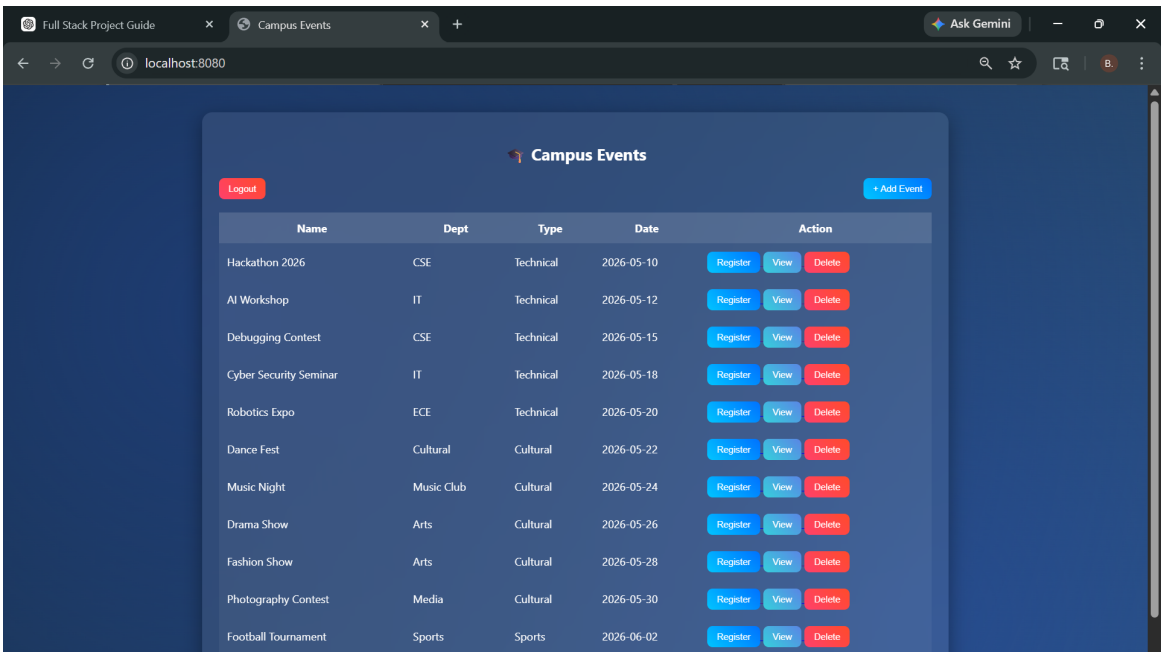

Figure 5.3: Event Dashboard

Figure 5.3 displays all available events with details such as name, department, type, and date. Users can register, view, or manage events based on their role.

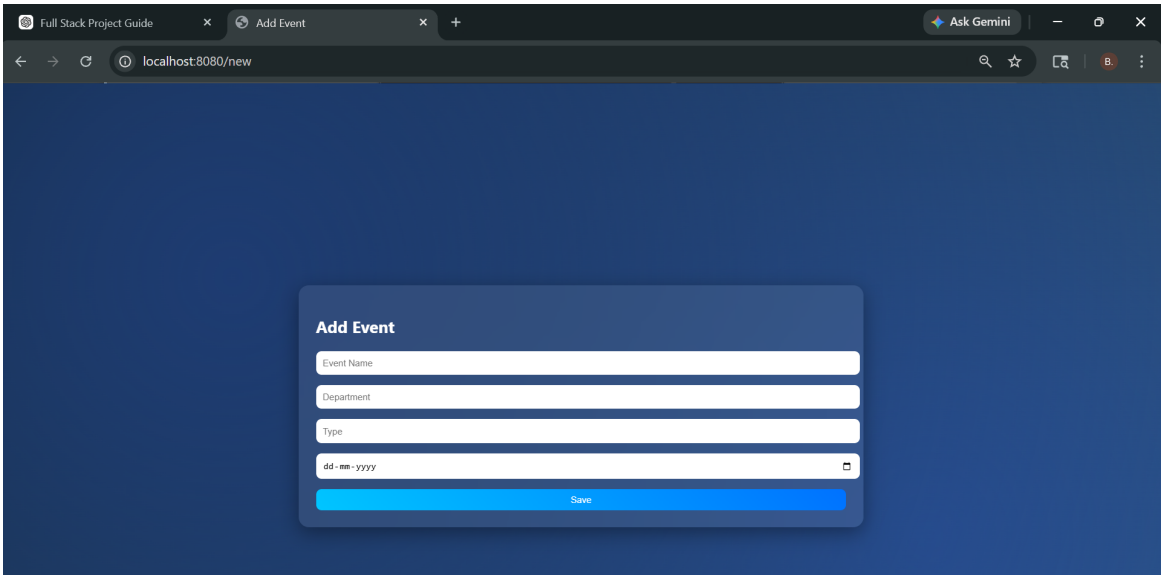

Figure 5.4: Add Event Interface

Figure 5.4 allows administrators to create new events by entering details such as event name, department, type, and date.

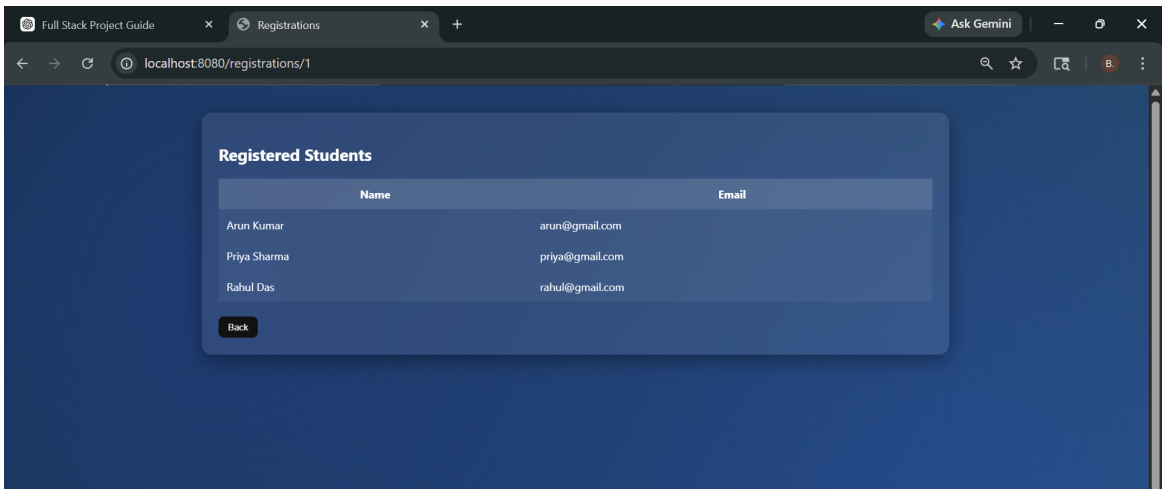

Figure 5.5: Registered Students View

Figure 5.5 shows the list of students registered for a particular event, including their names and email details.

5.3 Discussion

The implementation of the Smart Campus Event Management System demonstrates a clear improvement over traditional manual event handling methods used in colleges. The system integrates frontend, backend, and database components effectively to provide a seamless platform for managing campus events. This section discusses system efficiency, usability, performance, and architectural design.

5.3.1 Efficiency and Automation

The system automates key processes such as event creation, student registration, and participant tracking. Administrators can easily add, update, and delete events through a centralized interface, while students can register for events with minimal effort. This automation eliminates manual record-keeping, reduces paperwork, and minimizes human errors such as duplicate entries or incorrect data. The system also ensures better organization by handling multiple events simultaneously without

conflicts.

5.3.2 User Experience and Accessibility

The application provides a clean and responsive user interface developed using modern web technologies. Students can easily browse available events, view details, and register using simple actions. The admin interface is designed for ease of use, allowing quick access to event management features. The navigation between pages is smooth, ensuring a seamless experience. Overall, the system is accessible and user-friendly for both technical and non-technical users.

5.3.3 System Performance and Data Handling

The system ensures efficient data handling through a well-structured database and optimized backend processing. The use of RESTful APIs enables smooth communication between the frontend and backend layers. Data validation mechanisms are implemented to prevent issues such as duplicate registrations and invalid inputs. The system performs reliably under normal usage conditions, providing quick responses and accurate data retrieval.

5.3.4 Architectural Design and Scalability

The system follows a layered architecture consisting of presentation, application, and data layers. This separation of concerns improves maintainability and makes the system easier to understand and extend. The modular design allows new features such as notifications, analytics, or payment integration to be added in the future without affecting existing functionality. This makes the system scalable and suitable for larger deployments in real-world campus environments.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System successfully achieves its objective of providing a centralized and efficient platform for managing campus events and student participation. The system replaces traditional manual processes with a digital solution that simplifies event creation, registration, and monitoring.

The implementation demonstrates that using a full-stack architecture with a React-based frontend and a Spring Boot backend ensures smooth communication, efficient data handling, and a responsive user experience. Features such as authentication, event management, advanced filtering, and registration validation contribute to a reliable and user-friendly system.

Overall, the project improves operational efficiency, reduces manual errors, and enhances coordination between students and administrators. It serves as a scalable and practical solution for educational institutions to manage events effectively.

6.2 Future Enhancements

While the current implementation of the Smart Campus Event Management System provides a strong foundation for managing campus events and student registrations, the following enhancements are planned for future versions:

- **Real-Time Notification System:** Implementation of real-time notifications to inform users about event updates, reminders, and registration status using technologies like WebSockets.
- **Payment Integration:** Integration of secure payment gateways to support paid events and enable seamless online transactions.
- **QR Code-Based Attendance System:** Enhancing event participation tracking by implementing QR code-based check-in for faster and more accurate attendance management.
- **Advanced Analytics Dashboard:** Introducing detailed analytics for administrators, including participation trends, popular events, and department-wise insights for better decision-making.
- **Email and SMS Notification System:** Automated alerts for event confirmations, reminders, cancellations, and updates to improve communication and user engagement.
- **Role-Based Access Control Enhancement:** Expanding user roles such as Super Admin and Event Organizer to improve system security and provide better control over operations.
- **Mobile Application Development:** Developing a mobile application to improve accessibility and provide a seamless user experience across devices.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The Smart Campus Event Management System addresses a complex engineering problem by integrating frontend, backend, and database technologies to automate campus event management. It replaces manual processes with a scalable digital solution while handling constraints such as user authentication, event capacity, and real-time data processing. The system supports multiple stakeholders including students and administrators, ensuring efficient and reliable operation.The system involves real-time interaction between frontend interfaces and backend services, demonstrating the practical application of full-stack development concepts in solving real-world campus management problems.

Table 7.1: Mapping of Smart Campus Event Management System to CEP

Level	**Attribute**	**Characteristics**	**WKs**	**Justification**
WP1	Depth of knowledge	In-depth engineering knowledge	WK3–WK6	Uses React.js, Spring Boot, REST APIs, and H2 database for full-stack implementation.
WP2	Conflicting requirements	Technical vs non-technical conflicts	WK4–WK6	Balances user-friendly interface with constraints like event capacity and secure authentication.
WP3	Depth of Analysis	Analytical and design thinking	WK3–WK5	Designs relational schema linking students, events, and registrations with validation checks.
WP4	Familiarity	Partially new problems	WK4, WK8	Converts manual campus event handling into an automated web-based system with interactive dashboards and real-time data updates.
WP5	Applicable Codes	Limited standard solutions	WK6	Implements filtering, CRUD operations, and secure API-based workflows.
WP6	Stakeholder Needs	Multiple stakeholders	WK5–WK7	Supports students (registration) and admins (event management and analytics).
WP7	Interdependence	System integration	WK3, WK5, WK6	Integrates frontend, backend, and database for seamless real-time communication.

The table 7.1 maps the Smart Campus Event Management System to Complex Engineering Problem (CEP) attributes, highlighting its alignment with various knowledge, analysis, and design requirements. It demonstrates how the project integrates full-stack technologies, stakeholder needs, and system-level thinking to address real-world engineering challenges effectively.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Smart Campus Event Management System to CEA

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes React.js, Vite, Spring Boot, Spring Data JPA, and RESTful APIs.
EA2	Level of Interactions	Complex interactions between modules	Manages event lifecycle including creation, registration, and capacity validation.
EA3	Innovation	Application of modern technologies	Replaces manual systems with a Single Page Application (SPA) and automated workflows.
EA4	Consequences to Society and Environment	Impact on users and society	Reduces manual work, improves efficiency, and minimizes paper usage.
EA5	Familiarity	Beyond standard practices	Implements scalable architecture capable of handling multiple users and events.

The table 7.2 says that the project involves complex engineering activities such as system design, integration of multiple technologies, and implementation of scalable architecture. It demonstrates innovation by transforming traditional event management into a digital platform with real-time updates and efficient workflows.

7.3 SDG Mapping (CEP Section)

Table 7.3: SDG Mapping for Smart Campus Event Management System

SDG No.	**SDG Title**	**Relevance to the Project**
SDG 4	Quality Education	Encourages student participation in academic and extracurricular events, enhancing learning experience.
SDG 9	Industry, Innovation and Infrastructure	Implements modern web technologies to digitize campus event management.
SDG 11	Sustainable Cities and Communities	Supports a smart campus by providing an efficient and paperless system.
SDG 12	Responsible Consumption and Production	Reduces paper usage by digitizing event registrations and records.
SDG 17	Partnerships for the Goals	Enhances collaboration between students and administrators through a centralized platform.

The table 7.3 says that the Smart Campus Event Management System contributes to sustainable development by promoting digital transformation, improving accessibility to events, and reducing paper-based processes within educational institutions.

References

[1] Axios (2024), HTTP Client for API Requests. Available at: https://axios-http.com

[2] React (2024), Frontend UI Development Library. Available at: https://react.dev

[3] Vite (2024), Frontend Build Tool for Fast Development. Available at: https://vitejs.dev

[4] Spring Boot (2024), Backend Framework for Building RESTful APIs. Available at: https://spring.io/projects/spring-boot

[5] Spring Data JPA (2024), ORM Framework for Database Operations. Available at: https://spring.io/projects/spring-data-jpa

[6] MySQL (2024), Relational Database Management System. Available at: https://www.mysql.com

[7] Eventbrite (2024), Online Event Management and Ticketing Platform. Available at: https://www.eventbrite.com

[8] Meetup (2024), Platform for Organizing Events and Communities. Available at: https://www.meetup.com

[9] Townscript (2024), Event Registration and Management Platform. Available at: https://www.townscript.com